Comp90038
Tutorial 11:
Week 12

Today's Tutorial Flow

- Tutorial Questions
  - Floyd's Algorithm: Q79
  - Greedy Algorithm: Q80,81
  - Prim's Algorithm: Q82
  - Dijkstra Algorithm: Q83
  - Huffman tree :Q84

Recap

Dynamic programming

- algorithm design technique
- recurrence relation
- recursion involves overlapping instances

Simple instance: Fibonacci numbers

- rather than naive top-down algorithm
- solve and tabulate smaller sub-problems

Rather than solve overlapping sub-problems repeatedly

- bottom-up approach
- use tabulated results



Floyd's Algorithm:

Problem: All-Pairs Shortest-Paths

For weighted graphs with positive weights

Works for:

- directed and undirected graphs
- (sometimes works with non-positive weights, but not always)

Input:

- weight matrix W that holds all the edges' weights
- (for technical reasons, if there is no edge from node i to node j, we let $W[i,j] = \infty$)

We will construct the distance matrix D, step by step.

Same problem decomposition as in Warshall

- What is the shortest path from node i to node j using only nodes $\leq k$ as "stepping stones"?
- (assume nodes are numbered 1 to n)

use node k as a stepping stone, or we avoid it

- step from i to j using only nodes $\leq k-1$, or
- step from i to k using only nodes $\leq k-1$, and then step from k to j using only nodes $\leq k-1$.



Floyd's Algorithm

If G's weight matrix is W then we can express the recurrence relation for minimal distances as follows:

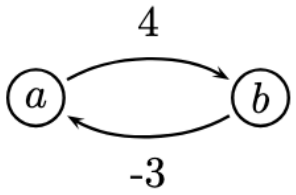
$$\begin{aligned} D_{ij}^{0} &= W[i,j] \\ D_{ij}^{k} &= min(D_{ij}^{k-1}, D_{ik}^{k-1} + D_{kj}^{k-1}) \end{aligned}$$

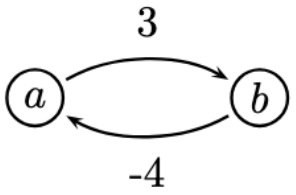
And then the algorithm follows easily:

```
function FLOYD(W[·,·], n)
    D ← W
    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                D[i,j] ← min(D[i,j], D[i,k] + D[k,j])
    return D
```

Tutorial Questions

Problem 1. Floyd's algorithm sometimes works even if we allow negative weights in a dag.



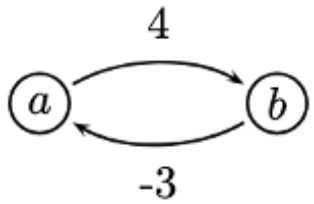


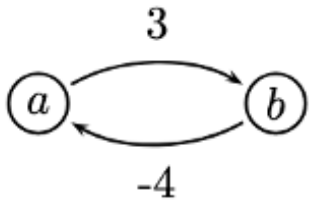
For example, for the left graph above, it will produce these successive distance matrices:

$$D^0 = D^1 = D^2 = \begin{bmatrix} 0 & 4 \\ -3 & 0 \end{bmatrix}$$

What happens for the right graph above? What do D^0, D^1 and D_2 look like? Explain why D^2 ends up giving an incorrect result in this case (but not in the previous case).

Problem 1. Floyd's algorithm sometimes works even if we allow negative weights in a dag.





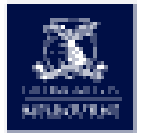
What happens for the right graph above? What do D^0, D^1 and D_2 look like? Explain why D^2 ends up giving an incorrect result in this case (but not in the previous case).

Solution. We get the following distance matrices:

$$D^0 = \begin{bmatrix} 0 & 3 \\ -4 & 0 \end{bmatrix} \quad D^1 = \begin{bmatrix} 0 & 3 \\ -4 & -1 \end{bmatrix} \quad D^2 = \begin{bmatrix} -1 & 2 \\ -5 & -2 \end{bmatrix}$$

This is where the algorithm stops, and the distances seem all wrong. The problem is that there is a *negative-weight cycle* in the graph. If we kept going round that cycle, we would get smaller and smaller negative "distances"—it does not make sense to ask for the path that has the smallest accumulated sum of weights.

If negative weights are possible, we really should add this test to Floyd's algorithm: If, at any point a negative element is created in the diagonal of the matrix, halt with some error message.



Greedy Algorithms

Make decisions based on locally best choice

Only sometimes best global solution



Suppose we have coin denominations 25, 10, and 1, and we want to change 35 cents using the smallest number of coins.

In general we will want to use as many 25-cent pieces as we can, then do the same for 10-cent pieces, and so on, until we have reached 35 cents. (In this case we use 25+10 cents.)

This greedy strategy will work for 35 cents but not always. For which of these it will not work? (a) 50 (b) 30 (c) 5.

Problem 2. We are given a sequence of "connection points" spaced out evenly along a straight line. There are n white, and n black points, in some (random) order.



The points are spaced out evenly, so that the distance between two adjacent points is 1.

The points need to be connected, so that each white point is connected to exactly one black point and vice versa. However, the total length of wire used must be kept as small as possible.

Consider the following (greedy) algorithm to solve the problem:

$k \leftarrow 1$
while there are still unconnected points **do**
    create all possible connections of length k
    $k \leftarrow k + 1$

Argue the correctness of this algorithm, or, alternatively, devise an example that proves that it may not produce an optimal wiring.

Problem 2. We are given a sequence of "connection points" spaced out evenly along a straight line. There are n white, and n black points, in some (random) order.



The points are spaced out evenly, so that the distance between two adjacent points is 1.

The points need to be connected, so that each white point is connected to exactly one black point and vice versa. However, the total length of wire used must be kept as small as possible.

Consider the following (greedy) algorithm to solve the problem:

$k \leftarrow 1$
while there are still unconnected points **do**
    create all possible connections of length k
    $k \leftarrow k + 1$

Argue the correctness of this algorithm, or, alternatively, devise an example that proves that it may not produce an optimal wiring.

Solution. The algorithm does not always yield the shortest wiring, witness the example:



This uses wire of length 10, but there are solutions that use only wire of length 8 (find one).

An efficient way of finding a best wiring makes use of a stack. The algorithm walks through the sequence of connection points in a linear fashion, starting with an empty stack. For each point it meets, it checks whether it has a point of the opposite colour on the top of the stack, and if so, the stack is popped, and the two points are connected. Otherwise it pushes the newly met point onto the stack.

Problem 3. Recall the definition of the knapsack problem. Given a set of items $S = \{i_1, i_2, \ldots i_n\}$ with

- weights: $w(i_1), w(i_2), \ldots, w(i_n)$
- values: $v(i_1), v(i_2), \ldots, v(i_n)$

and a knapsack of capacity W, find the most valuable selection of items that will fit in the knapsack. That is, find a set $I \subseteq S$ such that $\sum_{i \in I} w(i) \leq W$ and so that $\sum_{i \in I} v(i)$ is maximised.

Define the *benefit* of an item i to be the rational number $v(i)/w(i)$. Consider the following greedy approach to the problem:

Let $A[1]...A[n]$ hold the items from S, in decreasing order of benefit
$val \leftarrow 0$
$weight \leftarrow 0$
$k \leftarrow 1$
while $k \leq n \wedge weight + w(A[k]) \leq W$ **do**
    select $A[k]$
    $val \leftarrow val + v(A[k])$
    $weight \leftarrow weight + w(A[k])$
    $k \leftarrow k + 1$

That is, at each step, from the remaining items we simply pick the one that has the greatest benefit. Give a simple example to show that this greedy algorithm does not solve the knapsack problem.

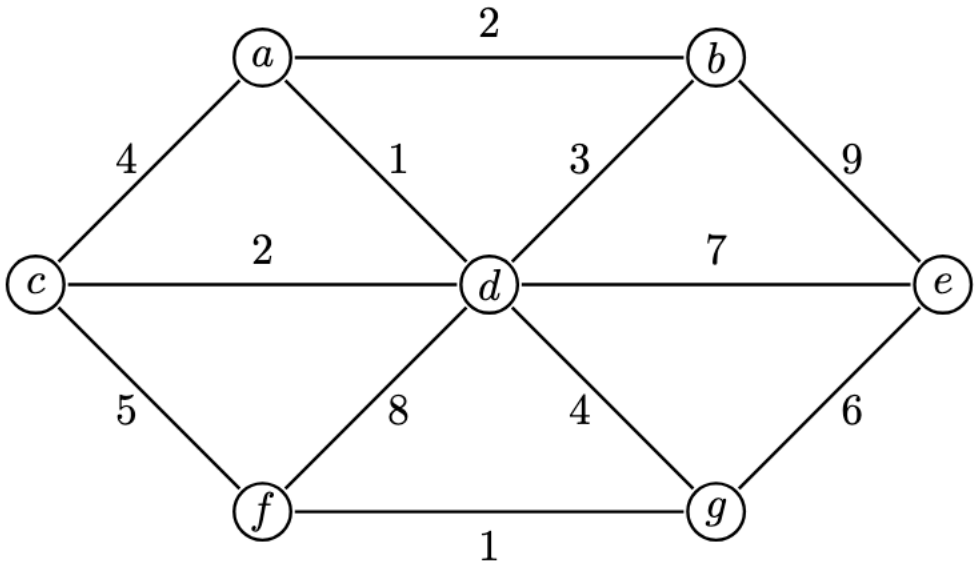
Let $A[1]...A[n]$ hold the items from S, in decreasing order of benefit
$val \leftarrow 0$
$weight \leftarrow 0$
$k \leftarrow 1$
while $k \leq n \wedge weight + w(A[k]) \leq W$ **do**
    select $A[k]$
    $val \leftarrow val + v(A[k])$
    $weight \leftarrow weight + w(A[k])$
    $k \leftarrow k + 1$

Solution. Assume capacity $W = 8$ and take, for example, three items with (value,weight) pairs as follows: i_1 : (6,5), i_2 : (4,4), i_3 : (3,4).

The three are already in decreasing order of benefit. If we pick i_1 greedily (since it has the highest benefit), there is room for no other item, so the value we achieve is 6. Of course, the better solution is to pick i_2 and i_3, for a total value of 7.

Tutorial Questions

Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?

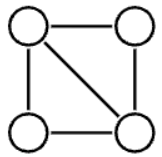


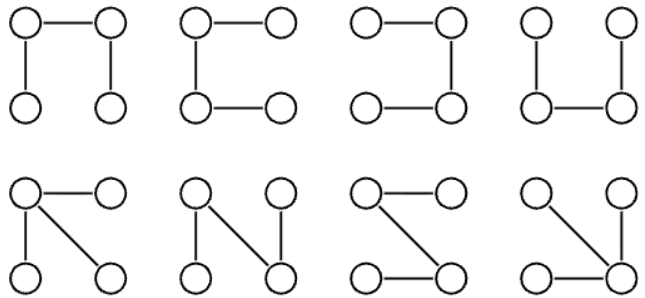
Definition: Tree

connected graph with no cycles

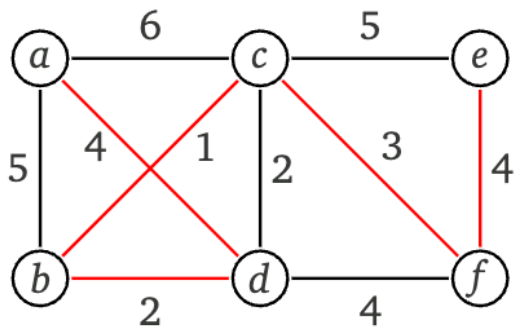
Definition: Spanning tree

A spanning tree of a graph $\langle V, E \rangle$ is a tree $\langle V, E' \rangle$ with $E' \subseteq E$.

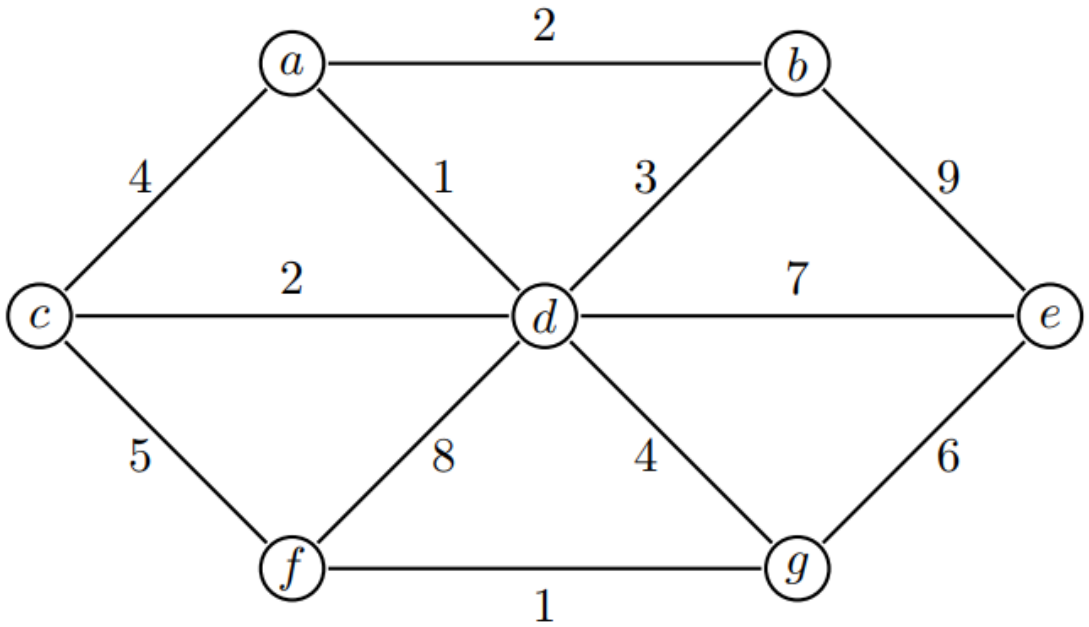
The graph  has eight different spanning trees:



Given a weighted graph, a sub-graph which is a tree with minimal weight is a minimum spanning tree for the graph.



Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?

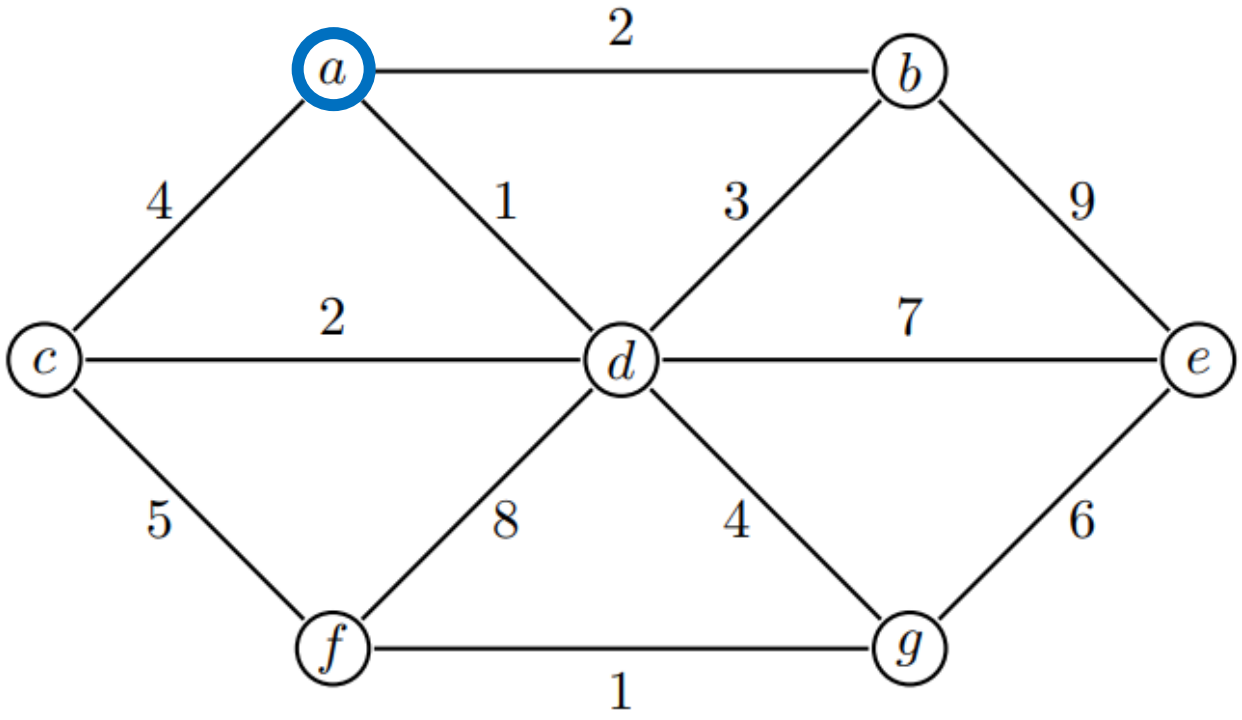


Greedy Algorithm

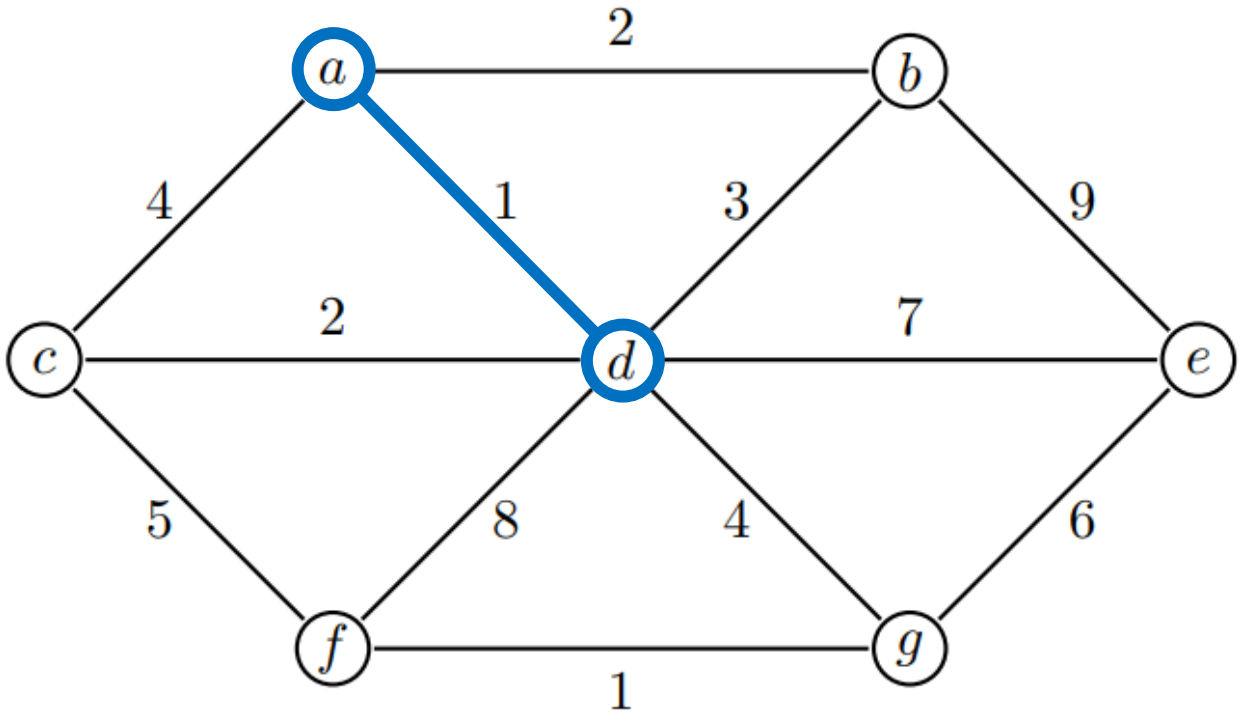
- sequence of subtrees T
- each adding a node together with an edge to a node in the previous subtree
- always pick a closest node from outside the tree

function PRIM($\langle V, E \rangle$)
  $V_T \leftarrow \{v_0\}$
  $E_T \leftarrow \emptyset$
  for $i \leftarrow 1$ to $|V| - 1$ **do**
    find a minimum-weight edge $(v, u) \in V_T \times (V \setminus V_T)$
    $V_T \leftarrow V_T \cup \{u\}$
    $E_T \leftarrow E_T \cup \{(v, u)\}$
  return E_T

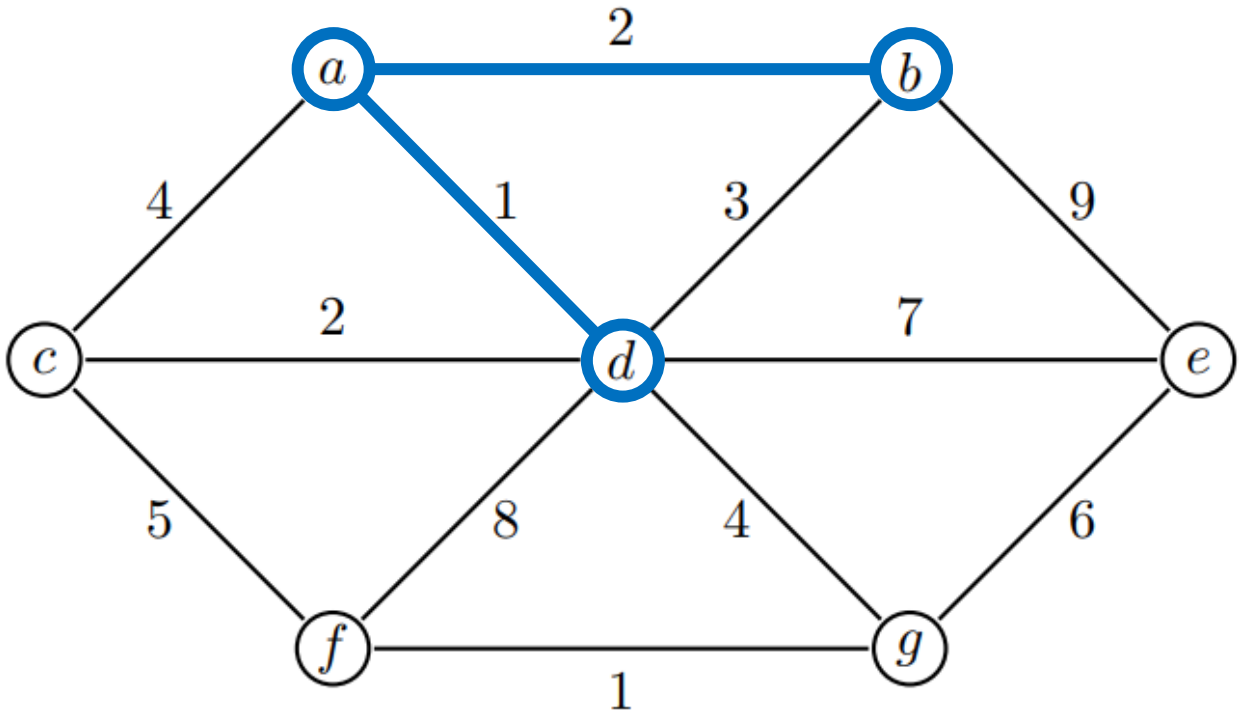
Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



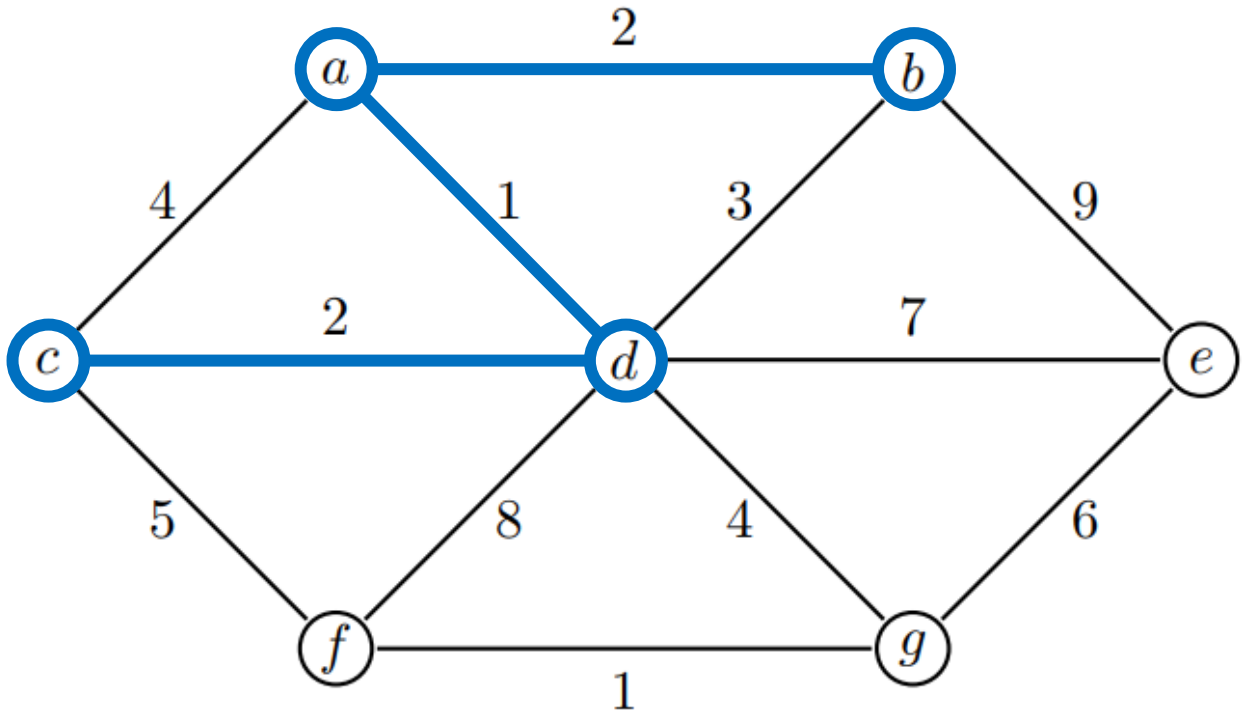
Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



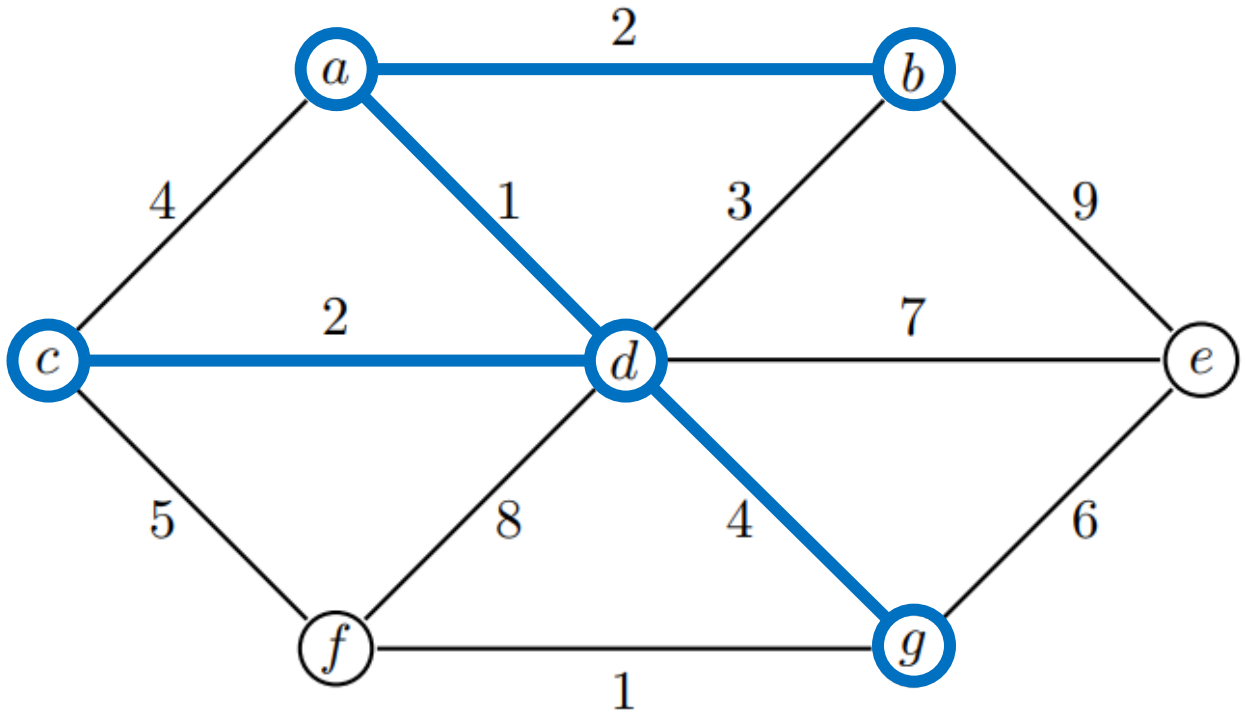
Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



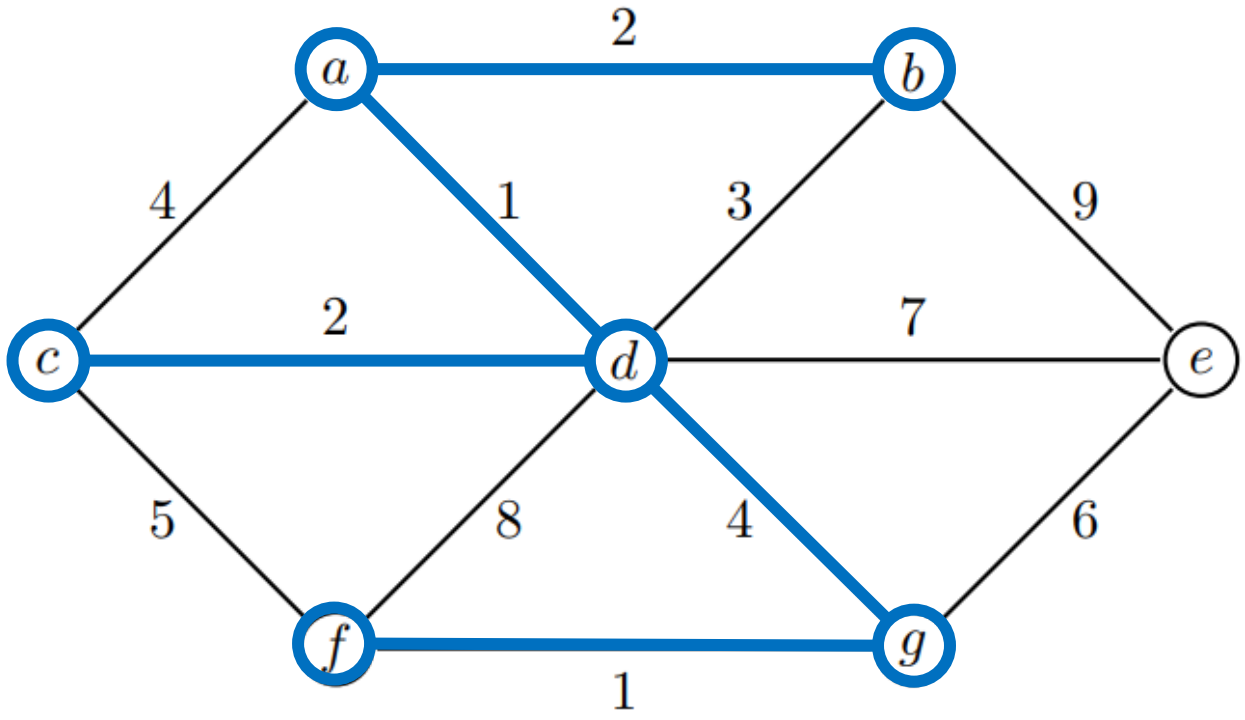
Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



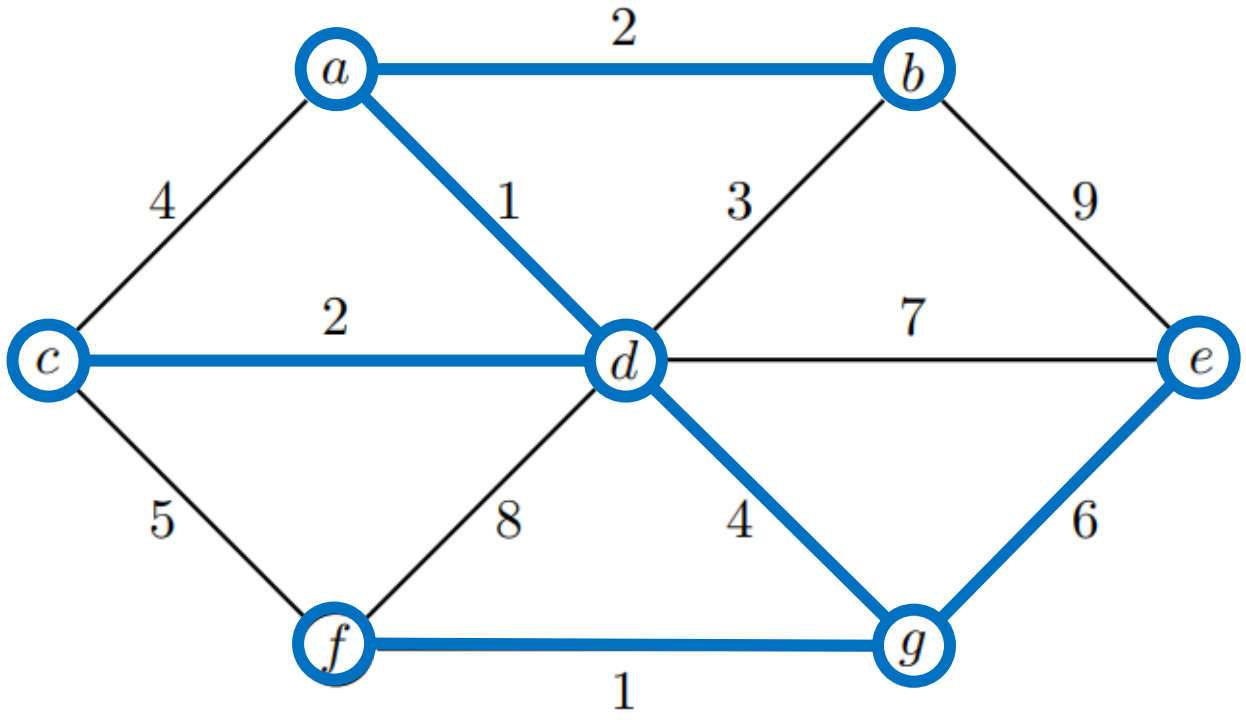
Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



Problem 4. Work through Prim's algorithm for the graph below. Assume the algorithm starts by selecting node a. Which edges are selected for the minimum spanning tree, and in which order?



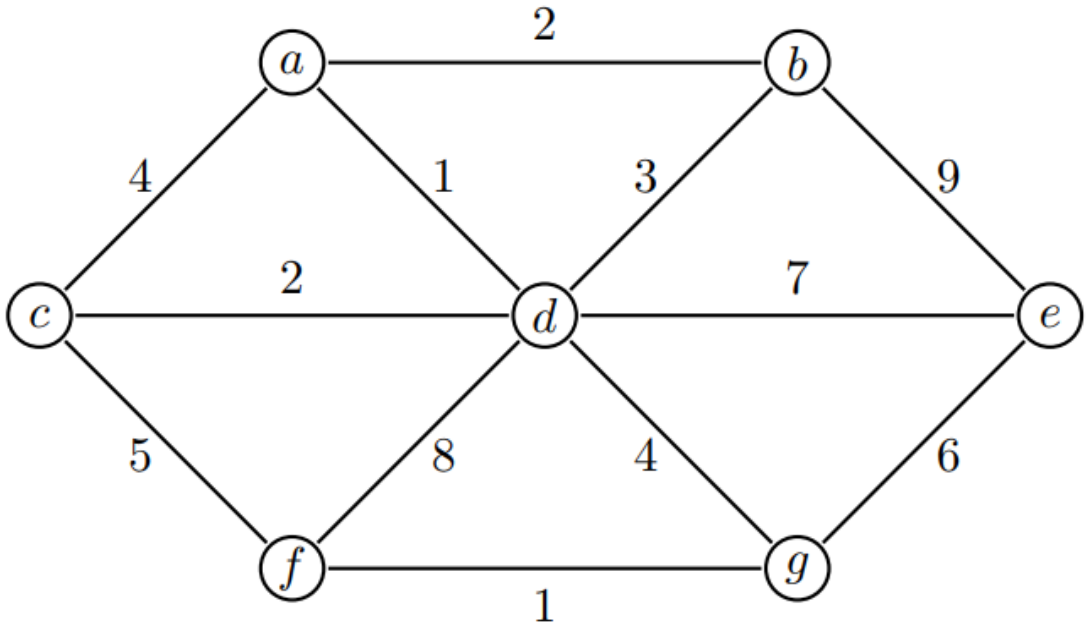
Greedy Algorithm

- sequence of subtrees T
- each adding a node together with an edge to a node in the previous subtree
- always pick a closest node from outside the tree

function PRIM($\langle V, E \rangle$)
  $V_T \leftarrow \{v_0\}$
  $E_T \leftarrow \emptyset$
  for $i \leftarrow 1$ to $|V| - 1$ **do**
    find a minimum-weight edge $(v, u) \in V_T \times (V \setminus V_T)$
    $V_T \leftarrow V_T \cup \{u\}$
    $E_T \leftarrow E_T \cup \{(v, u)\}$
  return E_T

function PRIM($\langle V, E \rangle$)
  for each $v \in V$ **do**
    $cost[v] \leftarrow \infty$
    $prev[v] \leftarrow nil$
  pick initial node v_0
  $cost[v_0] \leftarrow 0$
  $Q \leftarrow$ INITPRIORITYQUEUE(V) ▷ priorities are cost values
  while Q is non-empty **do**
    $u \leftarrow$ EJECTMIN(Q)
    for each $(u, w) \in E$ **do**
      if $weight(u, w) < cost[w]$ **then**
        $cost[w] \leftarrow weight(u, w)$
        $prev[w] \leftarrow u$
        UPDATE($Q, w, cost[w]$) ▷ rearranges priority queue

83. Use Dijkstra's algorithm to find the shortest paths for node e in the previous question's graph. That is, run the algorithm to determine the length of the shortest path from e to v, for all seven nodes v. Is the shortest path from e to b part of the graph's minimum spanning tree?



Dijkstra's Algorithm

- shortest-path algorithm from a fixed start node[1]
- (directed or undirected) weighted graphs
- complexity same as that of Prim's algorithm

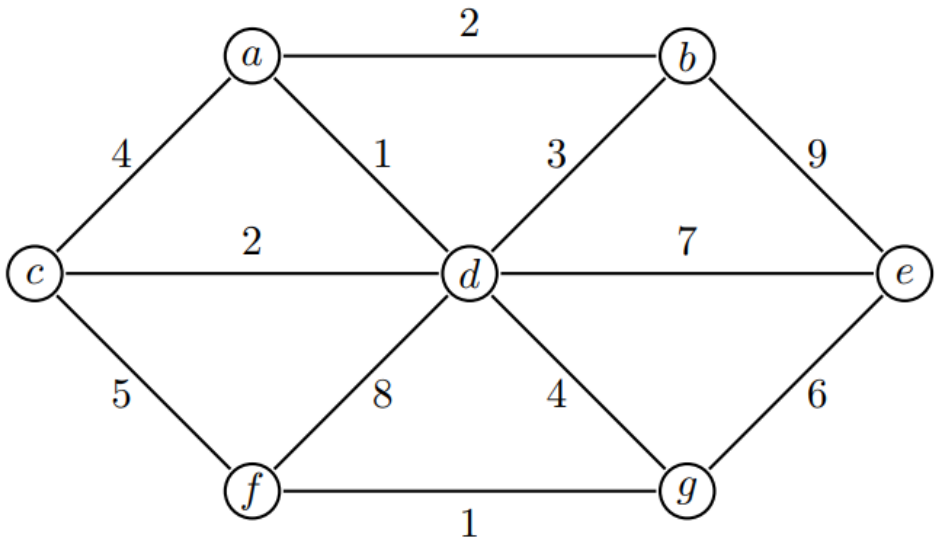
Note: Can not be applied to negative edge weights

83. Use Dijkstra's algorithm to find the shortest paths for node e in the previous question's graph. That is, run the algorithm to determine the length of the shortest path from e to v, for all seven nodes v. Is the shortest path from e to b part of the graph's minimum spanning tree?

```
function DIJKSTRA(⟨V, E⟩, v_0)
    for each v ∈ V do
        dist[v] ← ∞
        prev[v] ← nil
    dist[v_0] ← 0
    Q ← INITPRIORITYQUEUE(V)          ▷ priorities are distances
    while Q is non-empty do
        u ← EJECTMIN(Q)
        for each (u, w) ∈ E do
            if dist[u] + weight(u, w) < dist[w] then
                dist[w] ← dist[u] + weight(u, w)
                prev[w] ← u
                UPDATE(Q, w, dist[w])   ▷ rearranges priority queue
```

Select the vertex with the smallest distance from the source.

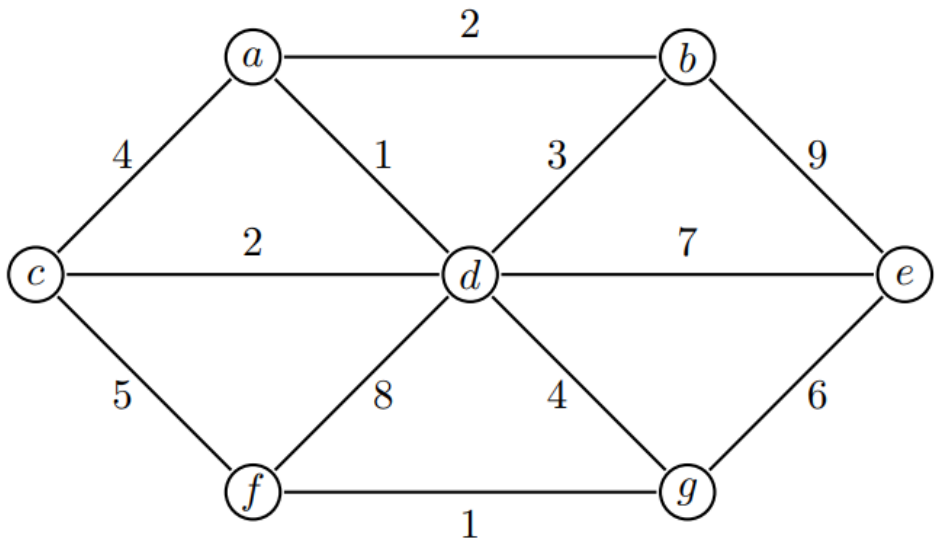
Note: Can not be applied to negative edge weights




```
function DIJKSTRA(⟨V, E⟩, v_0)
    for each v ∈ V do
        dist[v] ← ∞
        prev[v] ← nil
    dist[v_0] ← 0
    Q ← INITPRIORITYQUEUE(V)          ▷ priorities are distances
    while Q is non-empty do
        u ← EJECTMIN(Q)
        for each (u, w) ∈ E do
            if dist[u] + weight(u, w) < dist[w] then
                dist[w] ← dist[u] + weight(u, w)
                prev[w] ← u
                UPDATE(Q, w, dist[w])  ▷ rearranges priority queue
```

Covered	a	b	c	d	e	f	g
—	∞/nil	∞/nil	∞/nil	∞/nil	0/nil	∞/nil	∞/nil
e	∞/nil	$9/e$	∞/nil	$7/e$		∞/nil	$6/e$
e, g	∞/nil	$9/e$	∞/nil	$7/e$		$7/g$	
e, g, d	$8/d$	$9/e$	$9/d$			$7/g$	
e, g, d, f	$8/d$	$9/e$	$9/d$				
e, g, d, f, a		$9/e$	$9/d$				
e, g, d, f, a, b			$9/d$				
e, g, d, f, a, b, c							

83. Use Dijkstra's algorithm to find the shortest paths for node e in the previous question's graph. That is, run the algorithm to determine the length of the shortest path from e to v, for all seven nodes v. Is the shortest path from e to b part of the graph's minimum spanning tree?



Answer: The shortest paths from e are found to be:

a :	8
b :	9
c :	9
d :	7
e :	0
f :	7
g :	6

We already saw in the previous question that the e–b edge is not part of the graph's minimum spanning tree. These are the edges captured by the 'prev' relation:

84. Lemuel Gulliver wishes to compress the string "all␣big␣endians␣and␣all␣small␣endians". Help him by building a Huffman tree for the string (there may be several valid trees) and assign a binary code accordingly, to each of the eleven characters involved (we have used ␣ to make each space character visible). The frequencies are:

a	b	d	e	g	i	l	m	n	s	␣
6	1	3	2	1	3	6	1	5	3	6

How many bits are required for the encoded string?



Variable-Length Encoding

Idea

Assign characters codes in a way that common characters have short codes

Example

- "E" could have code 0
- No other character code can start with 0.

Property

- no character's code should be a prefix of some other character's code
- (unless we use separators, which would take up space)



Huffman Encoding: Choosing the Codes

Frequencies of letters

- We know them for common English text fairly well
- We can count all characters as a first step

How to assign codes to the characters once we know their frequencies?

Huffman's algorithm (1952)

- Fepeatedly select the two smallest weights and fuse them
- Another example of a greedy method.

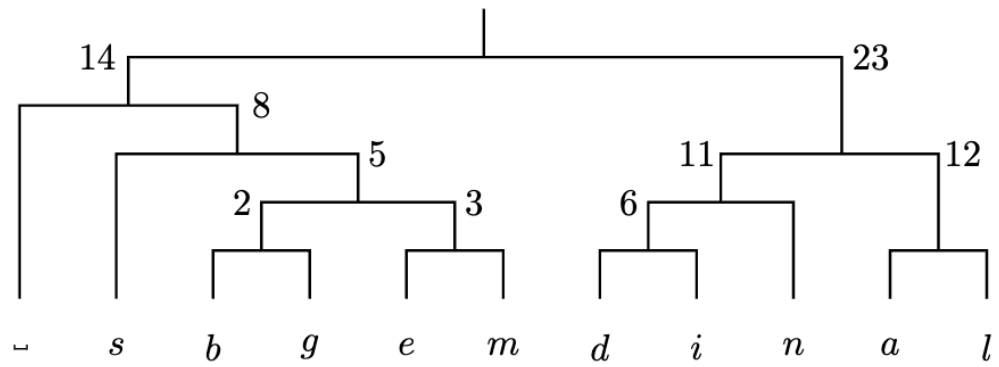
The resulting tree is a Huffman tree.

84. Lemuel Gulliver wishes to compress the string "all␣big␣endians␣and␣all␣small␣endians". Help him by building a Huffman tree for the string (there may be several valid trees) and assign a binary code accordingly, to each of the eleven characters involved (we have used ␣ to make each space character visible). The frequencies are:

a	b	d	e	g	i	l	m	n	s	␣
6	1	3	2	1	3	6	1	5	3	6

How many bits are required for the encoded string?

Answer: Different trees are possible. Here is one:



Based on this Huffman tree, we assign codes as follows:

a	b	d	e	g	i	l	m	n	s	␣
110	01100	1000	01110	01101	1001	111	01111	101	010	00

The encoded string is 11011111100011001001011010001110101100010011101010100011010110000011011111100010011111101111100011101011000100111010101010 and it consists of 121 bits.

Thank you for coming!

GOOD LUCK ON YOUR FINAL EXAM!!!